目录

- 1. 介绍
  - 介绍
  - 名词解释
  - 入门指南
  - 常见问题
  - Linux版本Docker环境
  - 常用接口
  - 示例
- 2. SD数据
  - 2.1 Link
  - 2.2 LinkNode
  - 2.3 Guidance
    - 2.3.1 车信LaneInfo
    - 2.3.2 电子眼Camera
    - 2.3.3 收费站TollStation
    - 2.3.4 警告牌WarningSign
    - 2.3.5 点限速PointSpeed
    - 2.3.6 方向看板Board
    - 2.3.7 长实线Lsl
    - 2.3.8 坡度Slope
- 3. HD数据
  - 获取HD数据图层
  - 3.1 LaneGroup

3.2 CrossArea
3.3 Lane
3.4 Boundary
3.5 SurfaceObject
3.5.1 导流区
3.5.2 RoadMark
3.5.3 TrafficLight
3.5.4 Pole
3.5.5 Obstacle
SurfaceObject总览
4. Object3DFeature
OBJECTS3D总览
模型索引
Object3DFeature
BuildingFeature
ModelFeature
TollStationFeature
压盖关系
LandMarkOverlapGroup
IndoorOverlapGroup
Texture
Model
Texture
Material
5. BMD数据
BMD总览

面类型要素
RegionFeature
SVGFeature
PedestrianOverpassFeature
点类型要素
PointFeature
SignFeature
ArrowFeature
TextLabelFeature
线类型要素
LineWithLabelFeature
LineFeature
6. 室内图BMD_INDOOR
室内图建筑信息IndoorBuildingInfo
室内建筑白名单IndoorWhiteInfo
室内建筑信息IndoorBuildingInfo
7. 附录
7.2 通用时间段表达

介绍

NERD-API: (Navigation Engine Resilient Database)-API

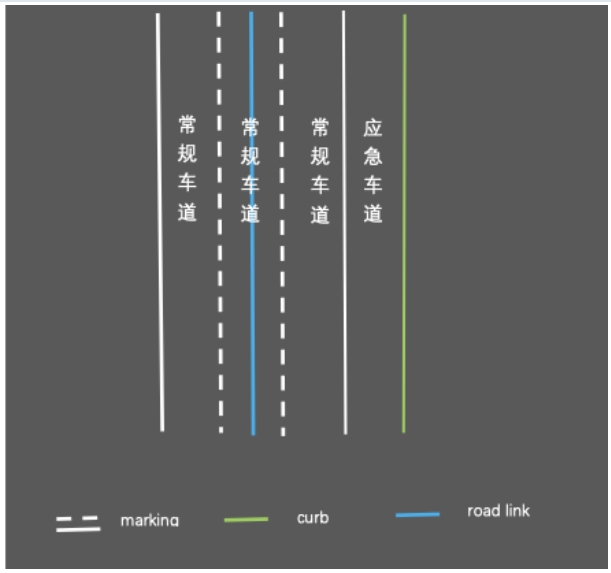
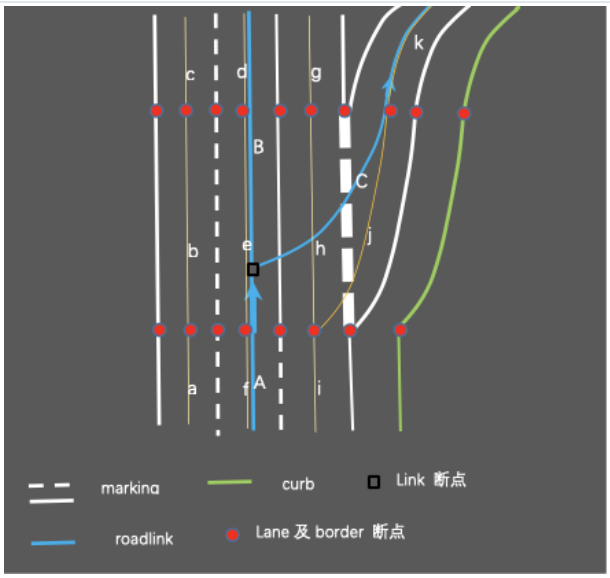
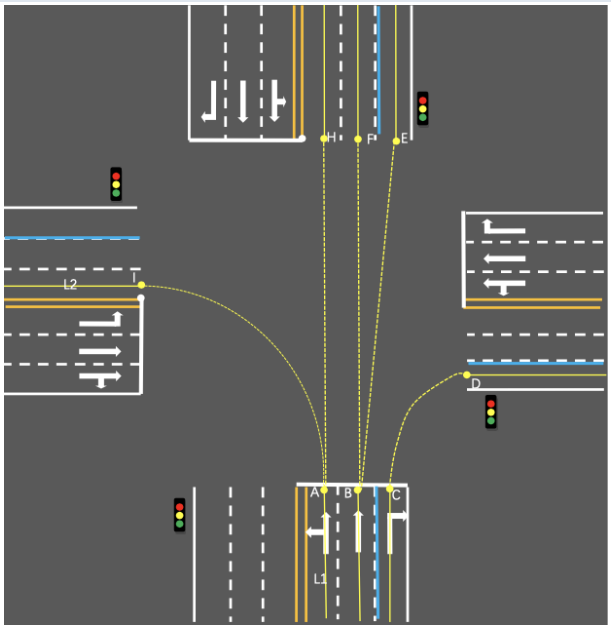
NERD-API 是一套面向自动驾驶业务的高精地图+标清地图数据引擎。

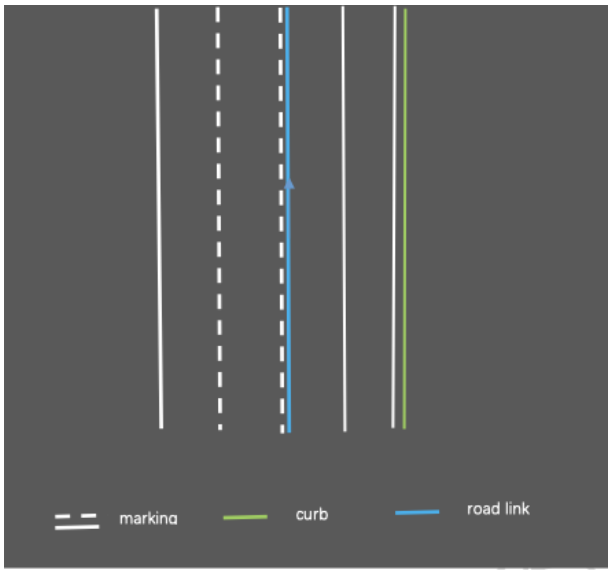
Features:

- 基于NDS规格，采用NDS.ev 开源zserio的定义编译产物，全面覆盖RoadModel和LaneModel结构
- 统一支持 **高精+SD** 地图格式，无缝切换
- 基于Google chromelem 项目，提供高性能异步查询接口

名词解释

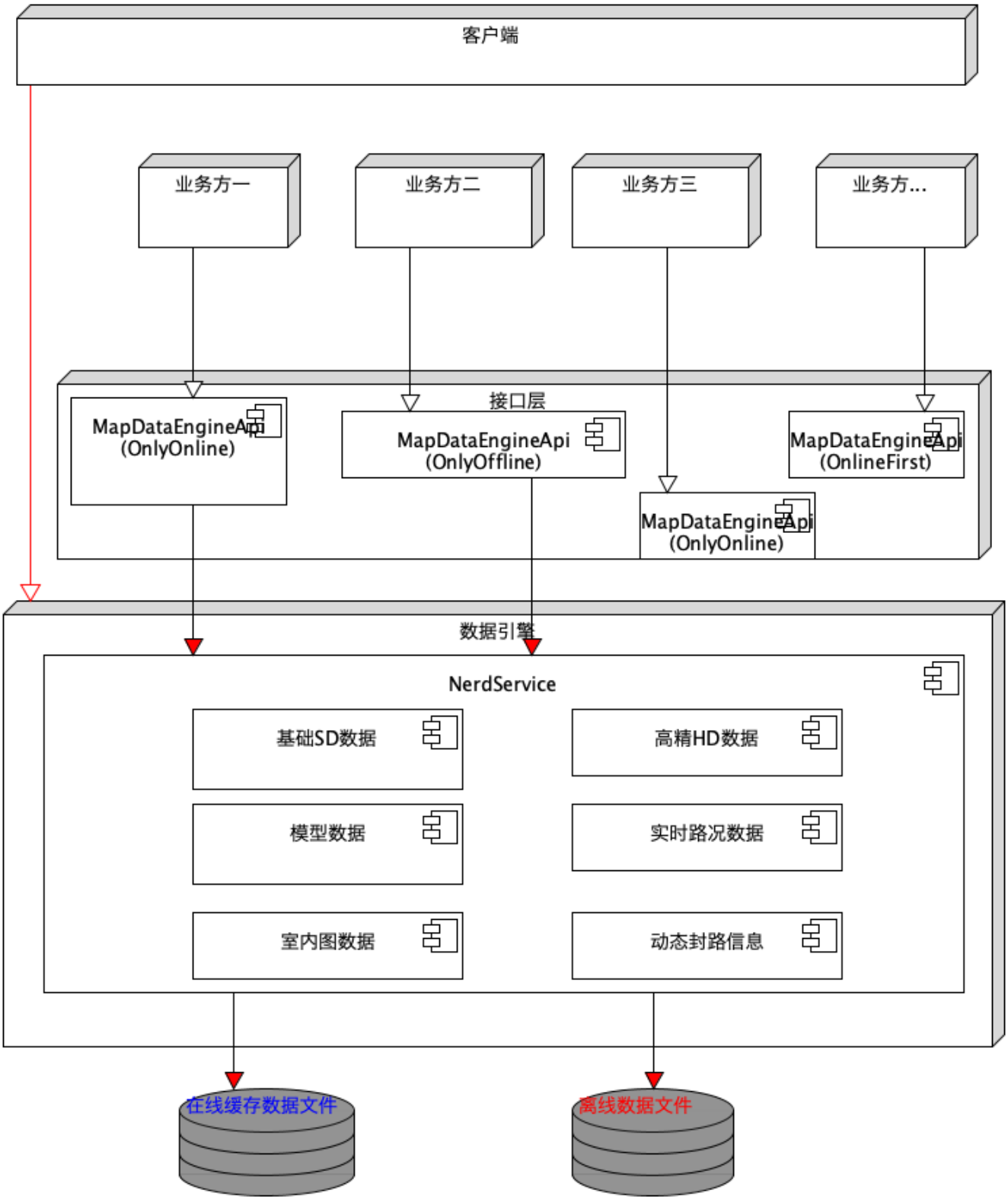
术语	英文释义	中文释义
SD Map	Standard Definition Map	标准清晰度地图，传统地图
HD Map	High Definition Map	高精度地图
HAD Map	Highly Automated Driving Map	高度自动驾驶地图

术语	释义	图例
Link	整个路面抽象出来的几何，供用sd中的道路link	
LaneGroup	每段道路内根据车道数变化、车道标线类型变化等因素，可能会被分为不同的LaneGroup(Section)	
Lane	车道中心线，位于该车道两个边界的几何中心，用于表达在该车道，建议车辆中心行驶过程中所在的位置	
LaneBoundary	车道边界是车道的几何表达，车道边界是车辆沿车道行驶时，行使空间的侧向边距。每一条lane都有两侧的边界	

		 marking curb road link

入门指南

架构设计



说明:

1. 使用引擎接口前，需要先完成引擎的初始化，传入必要的信息， 如 缓存数据路径，事件回调等;
2. 同时支持多个业务方创建多个引擎接口来加载地图数据;
3. 数据引擎内部没有网络能力，需要外部注入网络能力
4. 离线数据下载不在数据引擎模块内，需要使用离线数据，请联系我们项目接入离线下载模块;

下面介绍so+头文件的方式通过nerdapi获取nerd引擎数据。

初始化

首先，我们需要调用InitLogger函数完成Log配置以及调用InitGlobalConfig函数完成nerd引擎初始化。

```
/**
 * 全局配置类
 */
class NERD_EXPORT GlobalConfig {
 public:
  /**
   * 数据根目录，目录格式如下, 详细参考白皮书:
   *    db_root_path : 数据根目录
   *            |____ cache : 在线缓存路径
   *                    |___ bmd_xx.nds
   *                    |___ lane_xx.nds
   *            |____ offline  : 离线数据地址
   *                    |___ PRODUCT.NDS
   *                    |___ ROOT.NDS
   */
  std::string data_root_path{"data"};
  /**
   * HttpInterface 实例，用于在线数据下载
   */
  std::shared_ptr<mapbase::HttpInterface> http;
  /**
   * 离线调试模式，false时用离线城市包，true时用全国大文件，不支持Java层设置
   */
  bool offline_debug{false};
  /**
   * 在线缓存大小限制，超过限制时会清除缓存，单位：MB
   */
  int max_cache_size_mb{200};
  /**
   * 下发瓦片服务url
   * 备注: 内部默认会根据车图或手图平台设置对应的正式环境地址, 外部若需要指定环境,则由通过参数传入;
   */
  std::string tile_server_url{""};
  /**
   * 模型服务url
   * 备注: 内部默认会根据车图或手图平台设置对应的正式环境地址, 外部若需要指定环境,则由通过参数传入;
   */
  std::string model_server_url{""};
  /**
   * 路况服务下载地址
   * 备注:内部默认会根据车图或手图平台设置对应的正式环境地址, 外部若需要指定环境,则由通过参数传入;
   */
  std::string traffic_server_url{""};
  /**
   * 封路服务接口
   * 备注: 内部默认会根据车图或手图平台设置对应的正式环境地址, 外部若需要指定环境,则由通过参数传入;
   */
  std::string closed_road_server_url{""};
  /**
   * 客户渠道
```

```
  * 注意: 客户端必须注入,若不清楚渠道联系项目;
  */
  std::string channel{""};
  /**
  * 语言类型
  */
  MapDataLanguageType language_type{MapDataLanguageType::kLANGUAGE_CN};
  /**
  * 全局事件回调
  */
  std::shared_ptr<NerdEventListener> nerd_event_listener_holder_{nullptr};
};
```

```
#include "api/map_data_engine_api.h"


// 初始化log
nerd::api::InitLogger("./log", false);

// 先调用InitGlobalConfig, 只需要调用一次
nerd::api::GlobalConfig global_config;

// 设置config参数
global_config.data_root_path = "./data";
global_config.channel = "40006706701";

nerd::api::InitGlobalConfig(global_config);
```

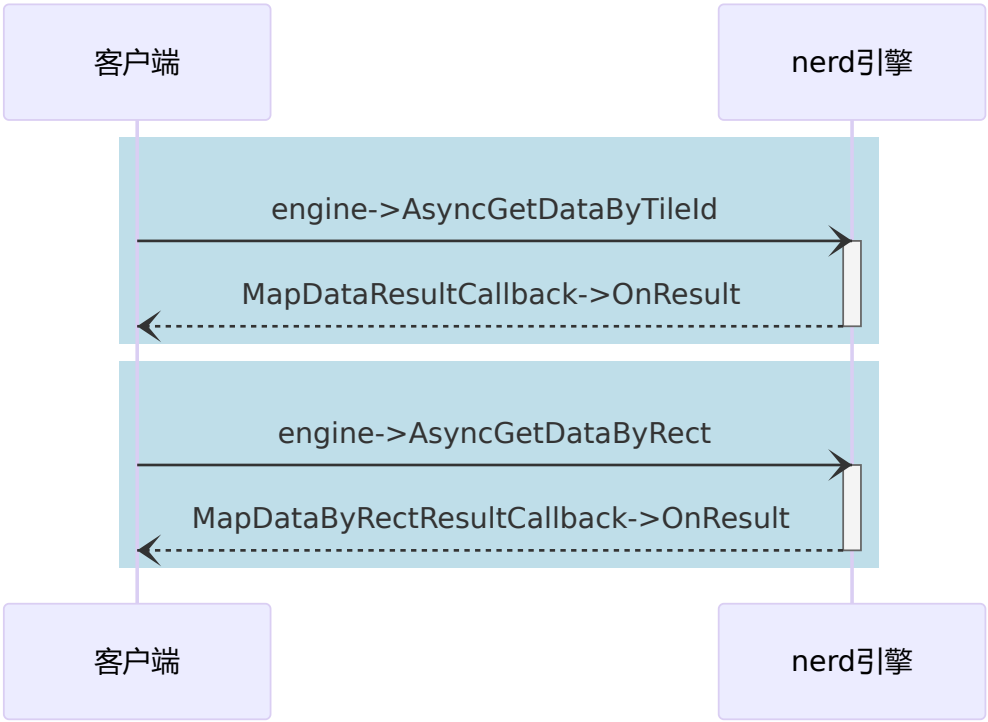
创建引擎

初始化之后，我们可以根据BuildingBlockID创建地图数据引擎

```
// 创建高精数据引擎
nerd::api::APIConfig lane_config;
lane_config.user_id = "lane";
lane_config.build_block_id = nerd::api::MapDataBuildingBlockID::kLANE;
auto lane_engine = nerd::api::CreateEngine(lane_config);

// 创建SD道路数据引擎
nerd::api::APIConfig route_config;
route_config.user_id = "route";
route_config.build_block_id = nerd::api::MapDataBuildingBlockID::kROUTE;
auto route_engine = nerd::api::CreateEngine(route_config);

// 创建基础渲染数据引擎
nerd::api::APIConfig bmd_config;
bmd_config.user_id = "bmd";
bmd_config.build_block_id = nerd::api::MapDataBuildingBlockID::kBMD;
auto bmd_engine = nerd::api::CreateEngine(bmd_config);

// 创建OBJECT3D数据引擎
nerd::api::APIConfig obj3d_config;
obj3d_config.user_id = "obj3d_config";
obj3d_config.build_block_id = nerd::api::MapDataBuildingBlockID::kOBJECTS_3D;
auto obj3d_engine = nerd::api::CreateEngine(obj3d_config);
```

读取图层数据

可以通过tileId或者矩形框的方式获取图层数据，推荐使用异步接口



根据BuildingBlockID创建的引擎从DataCube取图层数据接口的对应关系：

BuildingBlockID	取图层接口	元素举例	数据存放NDS tile层级
kLANE	GetHDLaneLayer	Lane信息等	15级
kROUTE	GetSDLinkLayer	Link信息等	13级
kBMD	GetBackgroundLayer	区域面, 河流, 文字等	3、5、7、10、13、15级
kOBJECTS_3D	GetObject3DLayer	小树/加油站, 楼块, 收费站等	15级

常见问题

欢迎使用nerdapi，在本文档中，我们将回答一些常见问题，帮助您更好地了解地图数据引擎的工作原理和使用方法。如果您在查阅这份文档之后依旧还有疑问，请随时联系我们的支持团队。

NOTE: 简单说明一下，nerdapi是一个库，用户可以查看include目录下的接口通览它能提供什么地图数据且nerdapi是仅仅是用来提供地图数据支撑功能的，nerdapi的用户通过nerdapi取出需要的地图数据，他们对这些数据进行处理和加工，然后为他们的用户提供地图相关的服务，nerdapi仅仅只提供地图数据，而不提供导航或者其他类型的服务。

获取不到数据怎么办?

- 首先检查tileId的层级是否正确，SD数据的tileId为13级，HD数据的tileId为15级，BMD数据的tileId可以是3、5、7、10、13、15级。
- 再检查初始化设置的引擎在/离模式是否正确，在线设置nerd::api::DataPreference::kOnlineOnly，离线设置nerd::api::DataPreference::kOfflineOnly
- 离线模式的离线文件路径是否正确，LANE.NDS存放路径GlobalConfig.data_root_path/lane/LANE.NDS，BGPOI.NDS存放路径GlobalConfig.data_root_path/BGPOI.NDS

nerd引擎返回的数据坐标怎么使用?

- nerd引擎返回的是NDS规范的球面坐标，结构为Coordinate，可以通过Coordinate::ToGeo()转换为gcj02坐标。

从导航SDK获取的SD路线ID如何调用的nerdapi获取link数据?

- 导航SDK获取的linkID是rawLinkId
- 得先取到rawId对应的tile或者rect数据，之后调用GetLinkByRawID取对应link。
  获取数据有两种方式:
  方式1.使用AsyncGetDataByTileId接口，沿路根据行点取tile，比如说从起点开始取tile，取到tile之数据后，遍历路线的rawLinkId,使用GetLinkByRawID接口，这个tile内的raw linkId对应的link都可以找到，出现找不到的就需要根据路径往后取tile，再重复这个过程;
  方式2.使用AsyncGetDataByRect接口，从起点开始取一个路线矩形框，遍历路线的rawLinkId，这个框内路线link可以使用GetLinkByRawID取到，再移动矩形框继续去沿途的link，需要注意矩形框不要太大。

是否有通过坐标获取所在道路ID的接口，如果没有，可以通过什么方式间接获取?

- 没有。可以用AsyncGetDataByRect获取矩形范围内的link集合，再从link集合中选择离指定坐标最近的link.

怎么获取全局规划的linkid集合?

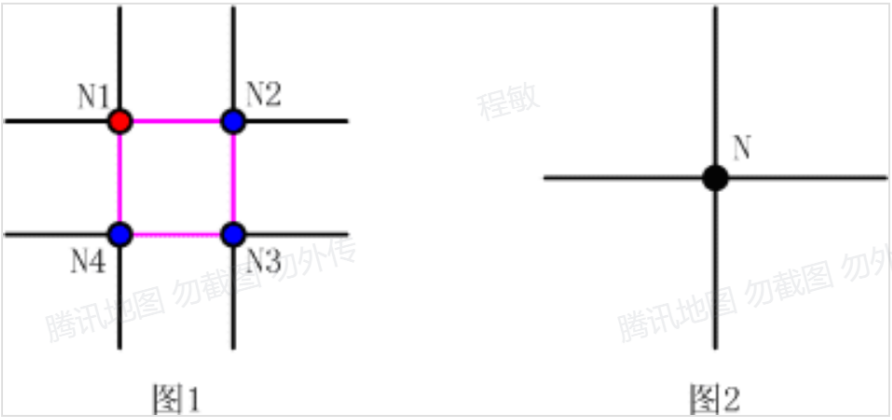
- 从导航SDK拿规划路径的linkid集合，从nerdapi拿不到全局规划的linkid集合。

ILinkNode是否对应的就是路口?

- LinkNode是link的端点，link在很多场景会打断，不一定是路口，比如一个端点可能只有一个入边一个出边（没有分歧）。

主Node和子Node能否提供示意图帮助理解，主Node和子Node之间是否也有一条link?

- 图1为复杂路口，一般有上下行分离link构成，路口有4个Node，其中一个为主Node，用来标识这个路口。
- 图2为简单路口，一般由非上下行分离link构成，路口只有一个交叉点。



两个Node之间是否有且只有一条link?

- 是的

双向道路的是对应一个道路id还是两个道路id？如果是两个道路id是否有提供正反向道路的关联关系?

- 一条双向通行的link只有一个Id

道路是否能区分主/辅道路，如果能区分是否有提供主道路与辅道路的关联关系?

- 可以区分主辅路，有主辅路属性，主辅路无关联关系，需要通过连通和位置关系判断

路口内道路是否有转向信息，例如直行，左转，掉头等？

- 有转向信息，路口处link的车信表达

一条link的车道数量是否是固定的？对于快驶入路口处的车道扩增导致车道数量变化怎么表达？

- 一条link上的车道数是固定的，在高精数据覆盖的区域车道数变化位置会打断link，普通link的车道数不一定准确，可能是在整个link占比较多车道数。

link->GetGeometry()取的link行点方向？

- GetGeometry是从起点到终点方向的行点列表

link打断规则

- 部分车道数变化位置、道路等级变化，道路类型变化、通行状态变化、道路名称等数据属性存在变化的场景会打断

SD数据是否可以获取路口处车道连通关系？

- SD数据没有车道连通关系，可以获取路口处link的车信信息，车信信息中有车道可通行的方向

如何获取link的车信？

- link->GetGuidance(nerd::api::GuidanceType::kLaneMetaInfo)，这样可以取到link的车信

SD数据linkID和rawLinkId有什么区别？

- linkID是tileId+tile内索引组成，rawLinkId为link原始id

坡度和曲率是什么级别的？

- 坡度，曲率是link级别的，不是lane级别的，不一定每条link都有坡度、曲率

车道数量变化会切分link吗？没有高精地图的地方，车道数量的精度有保障吗？

- 没有高精的地方车道数变化不会打断link，赋值link段内长度占比较多的车道数

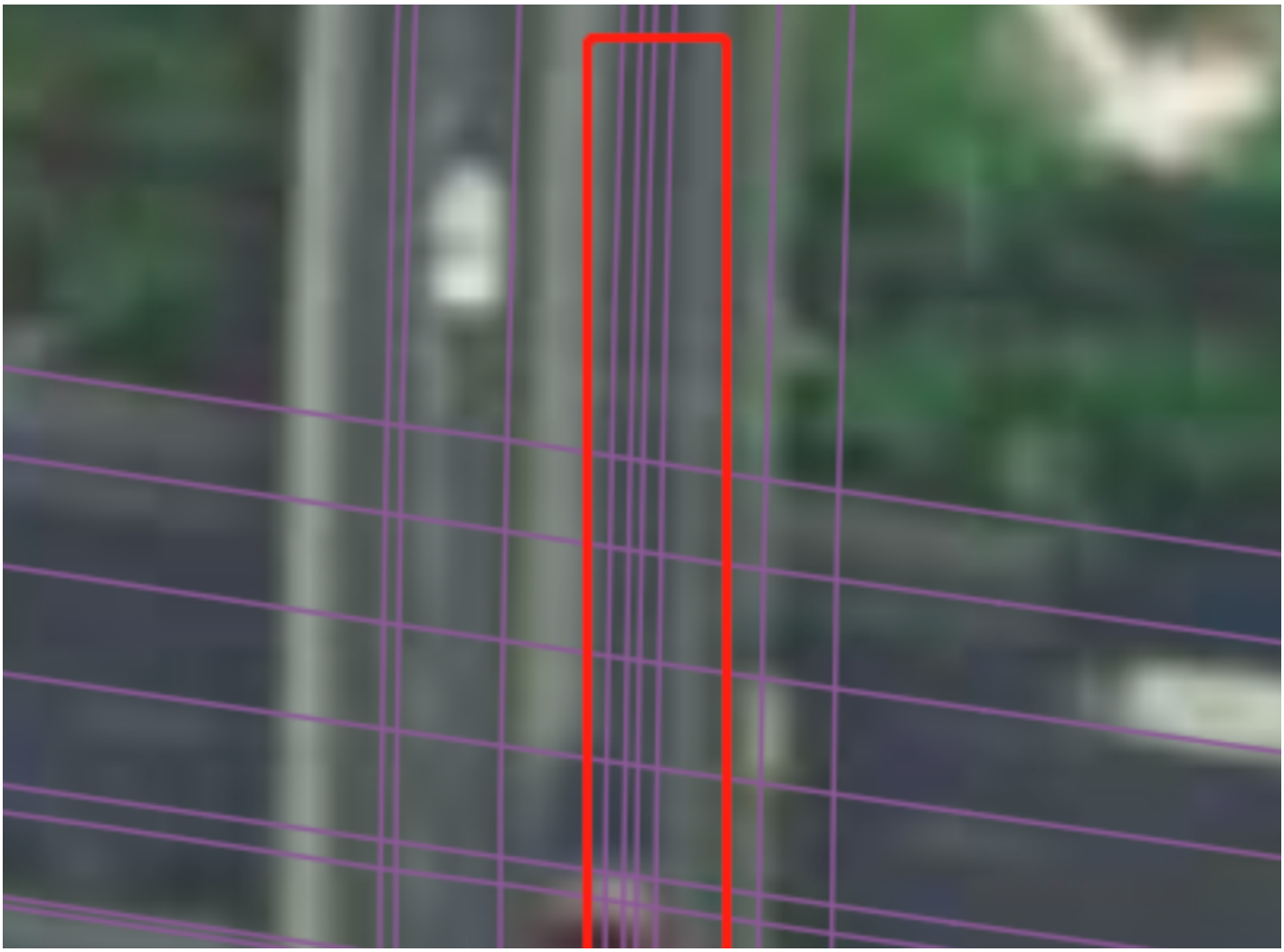
如果当前lane的nextLane跨tile了，如何获取？

- next是跨tile的时候，需要先获取NextlaneId， 根据NextlaneId里面的tileID， 再去获取相应的tile数据，之后再获取next， preLane同理。

HDmap中提取到的link数量要比SDmap中的link数量少很多，但是HDmap中的link拓扑连接更加丰富。这样理解是对的吗？

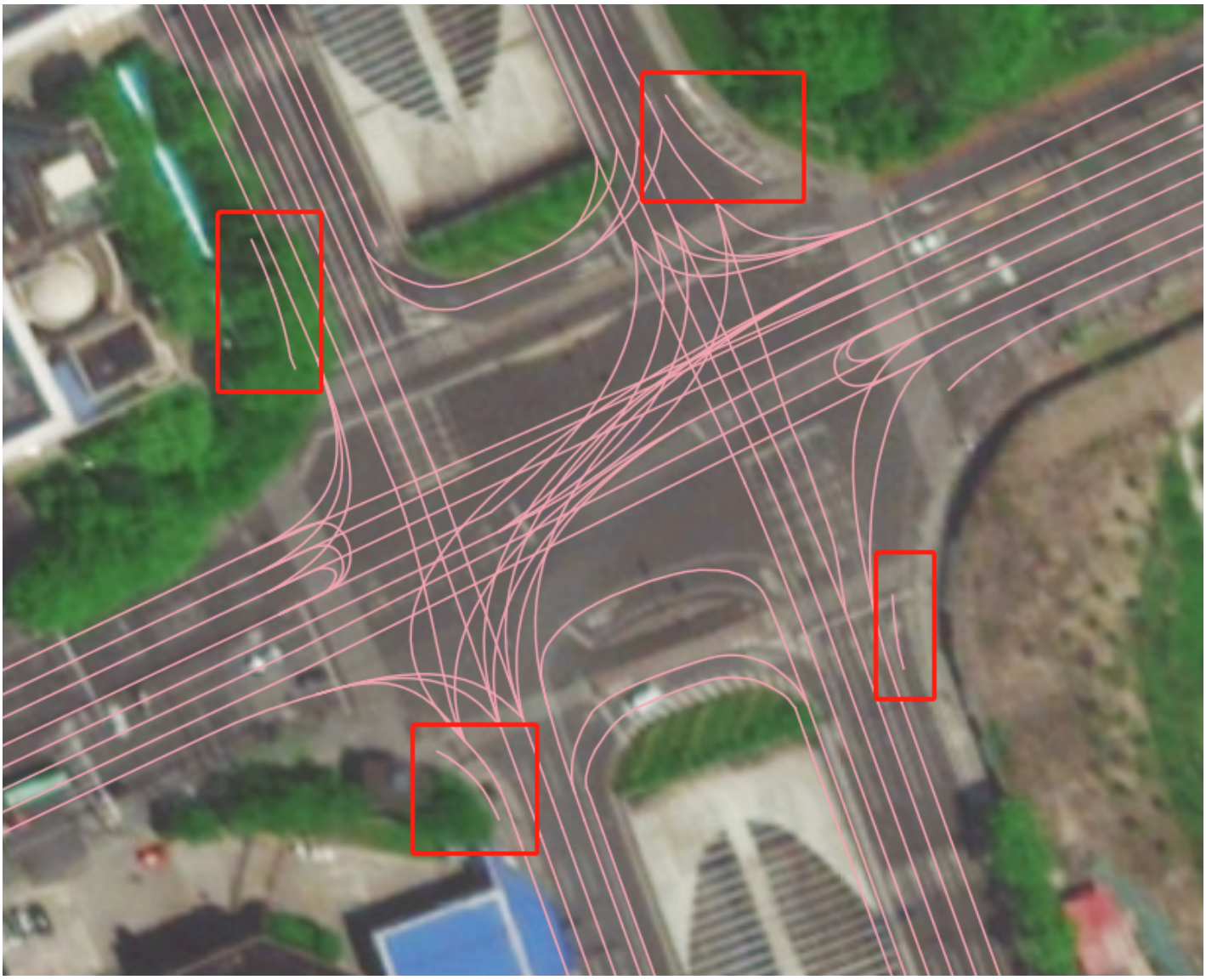
- hd比sd少时因为覆盖范围不一样，hd跟sd的连通表达方式不太一样，模型不一样，sd是用交叉口内部路，高精是单独的转向link

为什么下图中有些地方的boundary间隔特别小？



- 这个四条线分别归属上下行两个方向，每个方向两条线，分别是道路边缘和路肩线

为什么下图中Lane中心线附近有些线是孤立存在的？



- Lane中心线的情况，最外侧的那个是车道中心线是路肩的车道，车道的属性是路肩，本身路肩属性的车道在现实世界中是不连续的，所以存在不连通的情况。

SD Map 的link里是否有路口转向车道的顺序排列

- 通过link->GetGuidance(nerd::api::GuidanceType::kLaneInfo);可以取到车道转向信息

GetGuidance()返回的车道通行信息中，有个arrow，这个是怎么组合在一起的，如果link中有多条车道，怎么表示转向信息的？

- LaneTrafficInfo是个数组，表示多个车道。每个车道的箭头是由LaneArrowType与运算得到的，比如直行+右转的arrow = KLaneStraight | KLaneRight， 其车道是从左到右的顺序

nerdapi可以获取两点之间的路径吗

- nerd引擎不可以，需用导航SDK

84转02坐标算法是否固定

- 84转02坐标的算法及参数固定不变

Global Config 中的 channel 是什么意思？应该如何指定

- 渠道号，使用离线数据不用关注

LinkAttribute中的限速speed_limit_s2e/speed_limit_e2s，为什么会分为从起点到终点和从终点到起点？一条link会有两个行驶方向吗？

- link中存在双向道路，双向道路按划线方向，分别记录s2e和e2s的限速、车道数属性

在一条link上面，会不会有多个不同的限速？例如link前半段限速30km/h，后半段限速50km/h

- 数据中不会有，实际限速位置前后近距离有断点使用已有断点，距离已有断点过远时会打断link

link的几何形状返回的坐标点的疏密程度是怎样的？直线和弯道会不会有区别？

- 转弯位置为确保路线平滑形状点比直线密，但并没有规定必须密度范围是多少

LaneInfoMeta中的车道数lane_num和LinkAttribute中的车道数lane_num_sum是否一样？如果不一样，区别在哪

- link车道数以当前路段内较长的赋值，即如果link对应的实际车道数为3变4，3比4长，link车道数会给3；车信车道数按路口位置实际车道数赋值，3比4长，4是路口位置车道数，车信中会给4.

LaneInfoMeta中的车道数lane_num是否和lane_infos是绑定的，也就是lane_infos的size是否等于lane_num?

- ane_num和lane_infos的size是一样的

boundary 的 width 查出来为什么都是0?

- 车道边线目前没有宽度信息，值默认是0

Boundary的定义是什么?

- boundary是车道的分割标线，在车道数变化，SD道路断点，路口等位置会以道路面整组打断，沿道路方向存在属性变化时不打断，以区间形式返回，相同区间存在多个属性时，代表为多根线表达的分割线，如双白实线，双黄虚线等

Boundary区间的划分原则是什么？以及为什么一个boundary区间的属性要用vector 来存储，例如 BoundaryMarkingTypeVector

- boundary上的可能有多段不同的边线类型，比如前一段的标线是实线，后一段是虚线，因此boundary边界区间用std::vector<BoundarySetV2>。BoundarySetStyle里使用vector是因为同一段边线有可能有多种类型，比如双实线，boundary_types就是[MARKING,MARKING]

双实线的boundary_types是[MARKING, MARKING]，是否表示腾讯地图是把双实线这种复合线型，拆成两条 line 进行标注的？目前有无方法查询单条 line 的数据?

- 双实线等多线组合表达含义的线在数据标注时以一条线的形式表达，分别两条真实线的属性，坐标位置在两条线的中间，同区域段只有一个类型、颜色的线是单线

如果一个link正好在tile的边缘，那么通过link获取其前后继link，是不是存在获取不到的情况

- 是的，可以往后探路径上的tile，取到后面的tile再比较

LinkAttribute中的速度等级speed_class代表什么？和限速有什么区别?

- speed_class代表顺向限速的限速区间

通过矩形框取数据时，矩形框的坐标是否是火星坐标系经纬?

- 是的

矩形框的常规尺寸需要选择多大?

- 1～2公里的范围

是否有火星坐标系的GeoCoordinate类成员转为ENU坐标系的接口?

- 火星坐标系就是gjc02坐标系

是否存在火星坐标系的GeoCoordinate类成员 向Coordinate类成员转换的接口?

- 导航会下发导航路径的linkId列表(rawId)，取沿途的sd数据，通过GetLinkByRawID取到对应的sd link，再到link中取GetLaneGroupIds，根据LaneGroupId到hd中取高精数据
  gjc02转NDS坐标用FromGeoCoordinate

Link通过GetLinkCurvature接口获取的曲率和通过GetLinkPosition获取的曲率有何不同?

- GetLinkPosition已经废弃

Link通过GetGeometry接口获取的坐标点的个数和什么相关?

- link几何是按照道路实际形状进行绘制，点间隔只有最近距离的限制，形状点之间的距离不一致。

link获取的曲率正负如何定义，和车道方向是否相关?

- 沿着link的行驶方向，向左侧弯曲是正，向右侧弯曲是负

Link车道数目是否包含应急车道?

- 不包括应急车道的

如何获取导航服务下发路径的linkid list查找HD数据中的LaneGroup

- 导航下发的路径linkId列表中Id最低位表示方向，0表示从link的起点到终点方向，1则相反。linkId右移1位后得到rawId，取沿途的sd数据，通过GetLinkByRawID取到对应的sd link，再到link中取GetLaneGroupIds，根据LaneGroupId到hd中取高精数据

Linux版本Docker环境

Dockerfile 内容

```
## 基础镜像
FROM ubuntu:18.04

## 安装必要的软件包和工具
RUN apt-get update && \
    apt-get install -y curl g++ wget zip make && \
    wget -nv --content-disposition https://github.com/Kitware/CMake/releases/download/v3.20.1/cmake-3.20.1-linux-x86_64.tar.gz && \
    tar -xf cmake*.tar.gz -C /usr/local --strip-components 1 && \
    ln -s /usr/local/bin/cmake /usr/bin/cmake && \
    rm *.tar.gz
```

生成Docker镜像

将上图Docker内容保存到Dockerfile文件
在Dockerfile文件所在目录，执行如下命令生成Docker镜像

```
docker build -t ubuntu:18.04 .
```

使用Docker容器

将数据及引擎SDK拷贝到/data目录

```
// 生成容器nerdapi_dev，并将本地的data目录映射到nerdapi_dev容器的data目录
docker run -itd --name nerdapi_dev -v /data:/data ubuntu:18.04
// 进入nerdapi_dev容器
docker exec -it nerdapi_dev /bin/bash
```

进入nerdapi_dev容器的data目录，按照example/README.md的步骤编译运行示例程序

常用接口

1.坐标转换

1）简介

- WGS-84
  是一种用于地图学、大地测量学和导航（包括全球定位系统）的大地测量系统标准。WGS包含一套地球的标准经纬坐标系、一个用于计算原始海拔数据的参考椭球体，和一套用以定义海平面高度的引力等势面数据。
- NDS坐标
  将经度等分为232份，等分后映射到32位整形上，将纬度等分为231份，等分后也映射到32位整形上。
- GCJ-02
  俗称火星坐标系，国测局坐标。一种基于WGS-84制定的大地测量系统。

2）坐标转换

- NDS转GCJ-02

```
// 1. 使用 nerd::api::Coordinate的成员函数
auto ndsCoor = nerd::api::Coordinate{1391729057, 478852429};
auto gcj02Coor = ndsCoor.ToGeo();
std::cout << gcj02Coor.ToString() << std::endl;
// out: "[116.653382,40.13694693,0]"

// 2. 使用utils接口
auto ndsCoor = nerd::api::Coordinate{1391729057, 478852429};
auto gcj02Coor = nerd::api::utils::ToGeoCoordinate(ndsCoor);
std::cout << gcj02Coor.ToString() << std::endl;
// out: "[116.653382,40.13694693,0]"
```

- GCJ-02转NDS

```
nerd::api::GeoCoordinate gcj02Coor(116.653382, 40.13694693);
auto ndsCoor = nerd::api::utils::FromGeoCoordinate(gcj02Coor);
std::cout << ndsCoor.ToString() << std::endl;
// out: "lng=1391729057,lat=478852428,relativeZ=0"
```

2. 获取坐标所处TileID

一个坐标可以处于不同的层级（level），提供一个坐标和层级， 可以转换出这个坐标所在层级的TileID。

```
// 1. gcj02坐标
nerd::api::GeoCoordinate gcj02coor = {121.474879,31.223275};
auto tile_id1 = nerd::api::utils::GetTileIdByGeoCoordinate(gcj02coor, 13);
// tile_id1: 557040098

// 2. nds坐标
nerd::api::Coordinate ndsCoor(1449251757, 372508180);
auto tile_id2 = nerd::api::utils::GetTileIdByCoordinate(ndsCoor, 13);
// tile_id2: 557040098
```

3. 获取矩形框所处TileID

```
nerd::api::GeoCoordinate coor = {121.474879,31.223275};
double radio = 0.01;
nerd::api::GeoCoordinate p0(coor.x - radio, coor.y - radio);
nerd::api::GeoCoordinate p1(coor.x + radio, coor.y + radio);
nerd::api::Rect rect({p0, p1});
auto tile_ids = nerd::api::utils::GetTileIDSByRect(rect, nerd::api::MapLevel::kLEVEL_13);
// tile_ids: 557040053, 557040096, 557040055, 557040098
```

示例

本文档将介绍一下nerdapi的使用流程。

NOTE: 由于nerdapi支持在线和离线获取数据，所以在第2步初始化全局配置时，所配置的内容有所区别，下面会有说明。

1. 初始化日志

首先我们需要初始化日志，以便日后定位和追踪问题。

```
nerd::api::InitLogger("./log", false);
```

2. 初始化全局配置

用来保存磁盘缓存路径、服务器路径等一系列配置。

```
// 设置路径、服务器等
// NOTE: 对于离线获取数据的方式，只需要配置data_root_path以及offline_debug。
// TODO 对于在线获取数据的方式。
nerd::api::GlobalConfig global_config;
global_config.data_root_path = "./data";
global_config.tile_server_url = "http://sub.abc.com";
global_config.channel = "POC";

// 初始化全局配置，只需要调用一次
nerd::api::InitGlobalConfig(global_config);
```

3. 创建数据引擎

创建我们的数据引擎，以获取地图数据。

```
// 创建一个lambda函数，接收两个参数，地图数据类型以及获取方式。
  static auto create_engine =
      [](nerd::api::MapDataBuildingBlockID build_block_id, nerd::api::DataPreference dataPreference) ->
std::unique_ptr<nerd::api::MapDataEngineApi> {
    nerd::api::APIConfig api_config;
    api_config.user_id = "building_block:" + std::to_string(static_cast<int32_t>(build_block_id));
    api_config.build_block_id = build_block_id;
    api_config.preference = dataPreference;
    return nerd::api::CreateEngine(api_config);
  };

// 根据你需要的building block需要创建对应的引擎，这里创建了route和lane对应的引擎,分别使用了在线和离线方式。
  auto lane_api = create_engine(nerd::api::MapDataBuildingBlockID::kLANE,
nerd::api::DataPreference::kOnlineOnly);
  auto route_api = create_engine(nerd::api::MapDataBuildingBlockID::kROUTE,
nerd::api::DataPreference::kOfflineOnly);
```

4. 请求数据

当引擎创建好之后，请求需要的数据。这里有两种方式请求数据，一是根据TileID， 二是根据矩形框Rect。

由于nerdapi是异步请求数据的方式，所以用户需要实现Callback接口，当然也可以使用nerdapi已有的，这里使用nerdapi已有的。

不同的数据存在于不同的NDS tile层级，可以参考这个。

- 根据TileID

```
// 1. 根据TileID, 借用route的数据引擎演示
// 设置获取的TileID， 由于route的数据在第13层，你需要传入第13层级的TileID，否则获取不到数据。
nerd::api::GetMapDataBatchParam batch_param;
batch_param.tile_id_list.insert(tile_id);
// 设置Callback
auto route_tile_callback =std::make_shared<GetMapDataResultCallback>();
// 异步请求Tile数据
route_api->AsyncGetDataByTileId(route_batch_param, route_tile_callback);
// 等待结果
route_tile_callback->Wait();
```

- 根据矩形框

```
// 2. 根据矩形框请求数据， 借用lane的数据引擎演示
// 设置矩形框范围以及获取的数据层级
nerd::api::GetMapDataByRectParam l_param;
nerd::api::GeoCoordinate coor = {121.474879,31.223275};
double radio = 0.01;
nerd::api::GeoCoordinate p0(coor.x - radio, coor.y - radio);
nerd::api::GeoCoordinate p1(coor.x + radio, coor.y + radio);
nerd::api::Rect rect({p0, p1});
l_param.rect = rect;
// lane的数据在第15层，如果设置不正确则无法获取数据。
l_param.level = nerd::api::MapLevel::kLEVEL_15;
// 设置矩形框相关的Callback
auto l_callback = std::make_shared<GetMapDataByRectResultCallback>();
// 异步请求矩形框数据
lane_api->AsyncGetDataByRect(l_param, l_callback);
// 等待结果
lane_rect_callback->Wait();
```

5. 获取数据

Route数据

```
// 当结果获取之后，就可以获取其中的route或者lane相关的数据了。
// 1. Route数据
 const auto &route_callback_result = route_tile_callback->GetResult();
 // 判断是否存在数据
 if (route_callback_result->data_cube && route_callback_result->GetSDLinkLayer()) {
   auto layer = route_callback_result->data_cube->GetSDLinkLayer();
   // 获取所有的link
   auto allLinks = layer->GetLinkArr();
   // 获取....
 }
```

Lane数据

```
// 2. lane数据
const auto &lane_callback_result = lane_tile_callback->GetResult();
// 判断是否存在数据
if (lane_callback_result->data_cube && lane_callback_result->GetHDLinkLayer()) {
  auto layer = lane_callback_result->data_cube->GetHDLinkLayer();
  // 获取所有的lane
  auto allLanes = layer->GetLaneArr();
  // 获取所有的lanegroup
  auto allLanegroups = layer->GetLaneGroupArr();
  // 获取所有的SurfaceObject
  auto allSurfaceObjects = layer-> GetSurfaceObjects();
  // 获取...
}
```

6. 为您的用户提供地图服务

现在已经可以获取地图数据了，您可以对这些地图数据进行加工和处理，为您的用户提供地图服务。

> 完整代码可参考nerd-api/example下的在线和离线例子。

2.1 Link

下面介绍怎么获取link的属性、拓扑关系以及和link关联的信息。

获取link

可以通过link所在的tile或者矩形框获取。

```
// 通过link所在的tile获取
nerd::api::GetMapDataBatchParam param;
param.tile_id_list.insert(link_id.tile_id);
auto callback = std::make_shared<GetMapDataResultCallback>();
// 根据tile获取数据
auto job_id = map_data_engine_api->AsyncGetDataByTileId(param, callback);
callback->WaitUntilResponse(-1);
if (callback->resp_ && callback->resp_->data_cubes.size() > 0) {
  auto sd_layer = callback->resp_->data_cubes.at(0)->GetSDLinkLayer();
  if (sd_layer) {
    // 取到link_id对应的link
    auto link = sd_layer->GetLinkByID(link_id);

    // 获取tile中的所有link
    auto links = sd_layer->GetLinkArr();

    // 根据raw_id获取link
    auto link = sd_layer->GetLinkByRawID();
  }
}

// 通过link所在的矩形框获取
nerd::api::GetMapDataByRectParam param;
param.rect = link_rect;
auto callback = std::make_shared<GetMapDataByRectResultCallback>();
// 根据矩形框获取
auto job_id = map_data_engine_api->AsyncGetDataByRect(param, callback);
callback->WaitUntilResponse(-1);
if (callback->resp_ && callback->resp_->data_cubes.size() > 0) {
  auto sd_layer = callback->resp_->data_cubes.at(0)->GetSDLinkLayer();
  if (sd_layer) {
    // 取到link_id对应的link
    auto link = sd_layer->GetLinkByID(link_id);

    // 获取矩形框中的所有link
    auto links = sd_layer->GetLinkArr();

    // 根据raw_id获取link
    auto link = sd_layer->GetLinkByRawID();
  }
}
```

获取link的属性

通过GetAttribute可以获取link的道路名称、道路编号、道路等级、道路属性、长度、方向、车道数量等等属性，详情可以查看结构体LinkAttribute的定义。

```
// link_attr中lane_num_sum、lane_num_s2e、lane_num_e2s的值为99表示单车道过窄，0表示未调查，其他表示正常车道数
auto link_attr = link->GetAttribute();
```

获取link的几何形状

获取link的形状点，从起点到终点的坐标列表。

```
auto l_link_geos = link->GetGeometry();
```

获取link的Node

LinkNode

获取link的rawID

通过GetRawID获取，可以用于导航SDK下发道路ID的道路匹配。

```
auto l_link_raw_id = link->GetRawID();
```

获取坡度曲率信息

获取link的曲率信息。

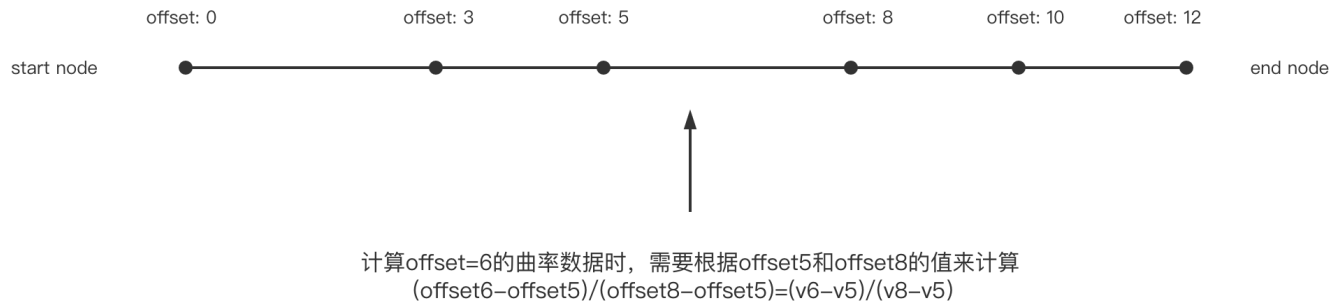
```
std::shared_ptr<const std::vector<LinkCurvature>> l_link_curvature = link->GetLinkCurvature();
```

```
struct LinkCurvature {
    int offset;
    int curvature;
}
```

LinkCurvature中，offset为从link起点开始的偏移量，单位为米，比如offset为3，则距离start node为3米；curvature为曲率分级值，分级标准如下：
分级标准：10^(-5)1/m，正负表示方向

分级标准	
……	
-154	[-0.00154,-0.00153)
……	
-1	[-0.00001,0)
0	[0,0.00001)
1	[0.00001,0.00002)
……	
154	[0.00154,0.00155)
……	

eg：



获取link的坡度，其中坡度分为纵向坡度和横向坡度。

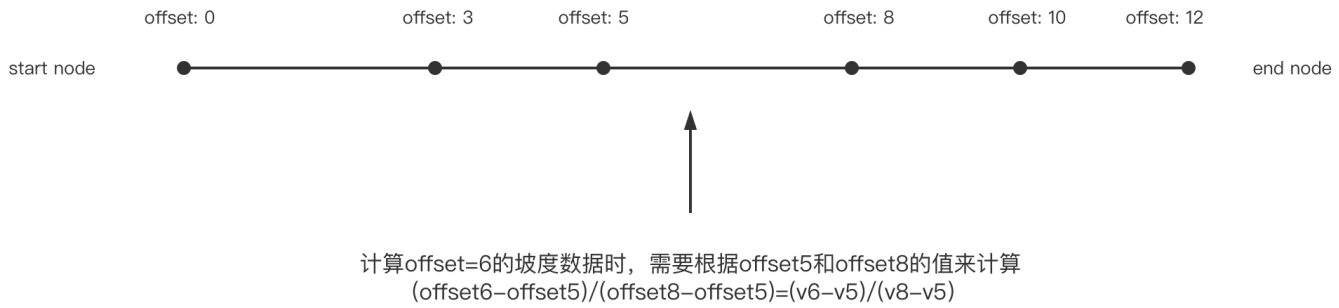
```
// 获取纵向坡度
std::shared_ptr<const std::vector<LinkSlope> l_link_vertical_slope = link->GetLinkVerticalSlope();
// 获取横向坡度
std::shared_ptr<const std::vector<LinkSlope> l_link_horizontal_slope = link->GetLinkHorizontalSlope();
```

```
struct LinkSlope {
    int offset;
    short slope;
}
```

同LinkCurvature类似，LinkSlope中offset为从link起点开始的偏移量，单位为米，比如offset为3，则距离start node为3米；slope为坡度分级值，分级标准如下：
分级标准：0.1°（度）；纵向坡度中，正数为上坡，负数为下坡；横向坡度中，正数为向左倾斜，负数为向右倾斜。

分级标准	
……	
-2	[-0.2°,-0.1°)
-1	[-0.1°,0°)
0	[0°,0.1°)
1	[0.1°,0.2°)
2	[0.2°,0.3°)
……	

eg：



获取link的警告牌信息

通过GetGuidance接口获取link的警告牌信息列表，记录该link包含的警告牌类型（如向左急弯、向右急弯、下陡坡、双向通行等类型，具体类型可查看TrafficSignType的定义）和坐标。

```
std::vector<std::shared_ptr<const IGuidanceInfo>> l_link_guidance = link->GetGuidance(nerd::api::GuidanceType::kWarningSign);
```

获取link的收费站信息

通过GetGuidance接口获取link的收费站列表，记录该link包含的收费站名称、收费方式（如现金、ETC、支付宝、微信等）、通道类型（如称重车道、绿色车道等）等信息。

```
std::vector<std::shared_ptr<const IGuidanceInfo>> l_link_guidance = link->GetGuidance(nerd::api::GuidanceType::kTollStation);
```

获取link的车信信息

通过GetGuidance接口获取link的车信列表，记录该link包含的箭头信息（如直行、左转、掉头等）、车道数量、车道类型等信息。

```
std::vector<std::shared_ptr<const IGuidanceInfo>> l_link_guidance = link->GetGuidance(nerd::api::GuidanceType::kLaneMetaInfo);
```

获取link的拓扑关系

通过GetNext和GetNextWithinIntersection获取link的后继link列表，其中GetNextWithinIntersection包含交叉点内link。LinkDirection表示搜索方向，只能为正向或逆向。

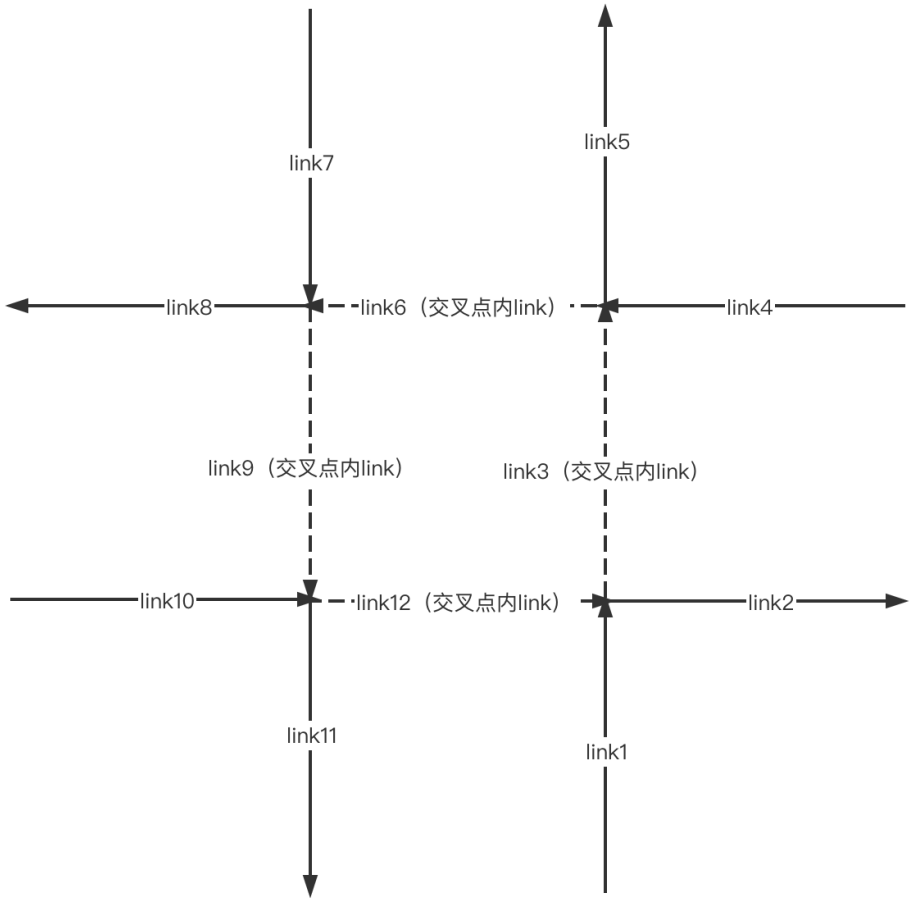
```
enum class LinkDirection : uint8_t {
  kUNKNOWN = 0,
  /**
   * 顺向，逆向
   */
  kBOTH_DIRECTION = 1,
  /**
   * 顺向
   */
  kSTART_TO_END,
  /**
   * 逆向
   */
  kEND_TO_START
};
```

```
auto l_link_next_links = link->GetNext(LinkDirection direction);
auto l_link_next_with_intersection_links = link->GetNextWithinIntersection(LinkDirection direction);
```

通过GetPrevious和GetPreviousWithinIntersection获取link的前驱link列表，其中GetPreviousWithinIntersection包含交叉点内link。

```
auto l_link_prev_links = link->GetPrevious(LinkDirection direction);
auto l_link_prev_with_intersection_links = link->GetPreviousWithinIntersection(LinkDirection direction);
```

eg:



从link1出发，调用link1->GetNext(nerd::api::LinkDirection::kSTART_TO_END)，返回的结果为link2、link5、link8、link11
调用link1->GetNextWithinIntersection(nerd::api::LinkDirection::kSTART_TO_END)，返回的结果为link2、link3

获取link的交通限制信息

通过GetRestrictions接口可以获取到link的交通限制信息，其中包含进入linkid和退出linkid。

```
auto l_link_restrictions = link->GetRestrictions();
```

获取link对应的HD数据LaneGroupId

通过GetLaneGroupIds接口可以获取link对应在HD数据中的laneGroupId，由于双向link可能会对应不同的laneGroup，所以需要道路方向的参数。

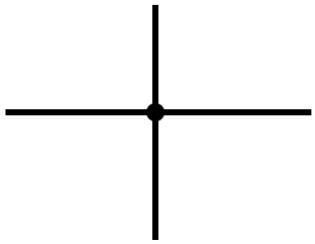
```
auto l_link_lanegroup_id = link->GetLaneGroupIds(LinkDirection direction);
```

2.2 LinkNode

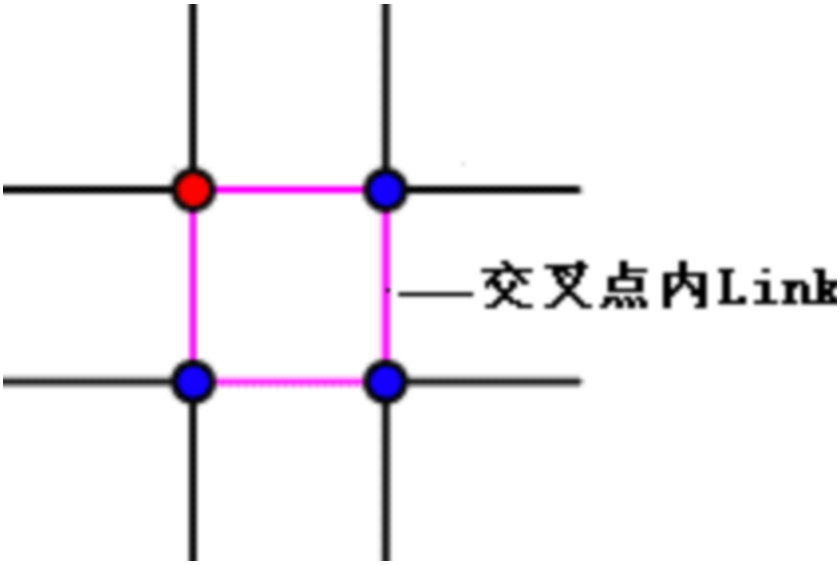
LinkNode是指link的端点。一条link有两个端点，其中一个是起点，一个是终点。link形点列表第一个点的坐标和起点重合，形点列表最后一点的坐标和终点重合。

LinkNode类型

LinkNode类型有非路口节点、单一路口节点、复合路口主点、复合路口子点四种。
单一路口节点是指非上下行link相交的路口，如下图所示：



复合路口一般是含有上下行分离link的路口，如下图所示，图中红色Node是复合的主点、蓝色Node是复合路口的子点，一个复合路口只有一个主点。



下面介绍获取linkNode的相关信息。

获取LinkNode

可以通过link对象直接获取link的起点和终点，或者通过NodeIDType类型的node_id获取。

```
// 通过link对象获取link的起终点
auto start_node = link->GetStartNode();
auto end_node = link->GetEndNode();

// 通过nodeId获取,先获取node所在的tile数据，再用nodeId查找对应的node
nerd::api::GetMapDataBatchParam param;
param.tile_id_list.insert(node_id.tile_id);
auto callback = std::make_shared<GetMapDataResultCallback>();
auto job_id = map_data_engine_api->AsyncGetDataByTileId(param, callback);
callback->WaitUntilResponse(-1);
if (callback->resp_ && callback->resp_->data_cubes.size() > 0) {
  auto sd_layer = callback->resp_->data_cubes.at(0)->GetSDLinkLayer();
  if (sd_layer) {
    // 取到node_id对应的node
    auto node = sd_layer->GetLinkNodeByID(node_id);
  }
}
```

获取LinkNode的属性

```
// 获取是否有红绿灯
bool traffic_flag = node->GetTrafficLightFlag();

// 获取node坐标
auto coor = node->GetPosition();

// 获取交叉口路口标识
auto cross_flag = node->GetCrossFlag();
if (cross_flag == nerd::api::CrossFlag::kSubNode) {
  // 获取主点
  auto main_node = node->GetMainNode();
} else if (cross_flag == nerd::api::CrossFlag::kMainNode) {
  // 获取子点
  auto sub_nodes = node->GetSubNodeIds();
}
```

获取linkNode的Guidance信息

部分信息是挂接在linkNode上的，比如方向看板、长实线、坡度。

```
// 获取方向看板
node->GetGuidance(nerd::api::GuidanceType::kBoard);

// 获取长实线
node->GetGuidance(nerd::api::GuidanceType::kLsl);

// 获取坡度
node->GetGuidance(nerd::api::GuidanceType::kSlope);
```

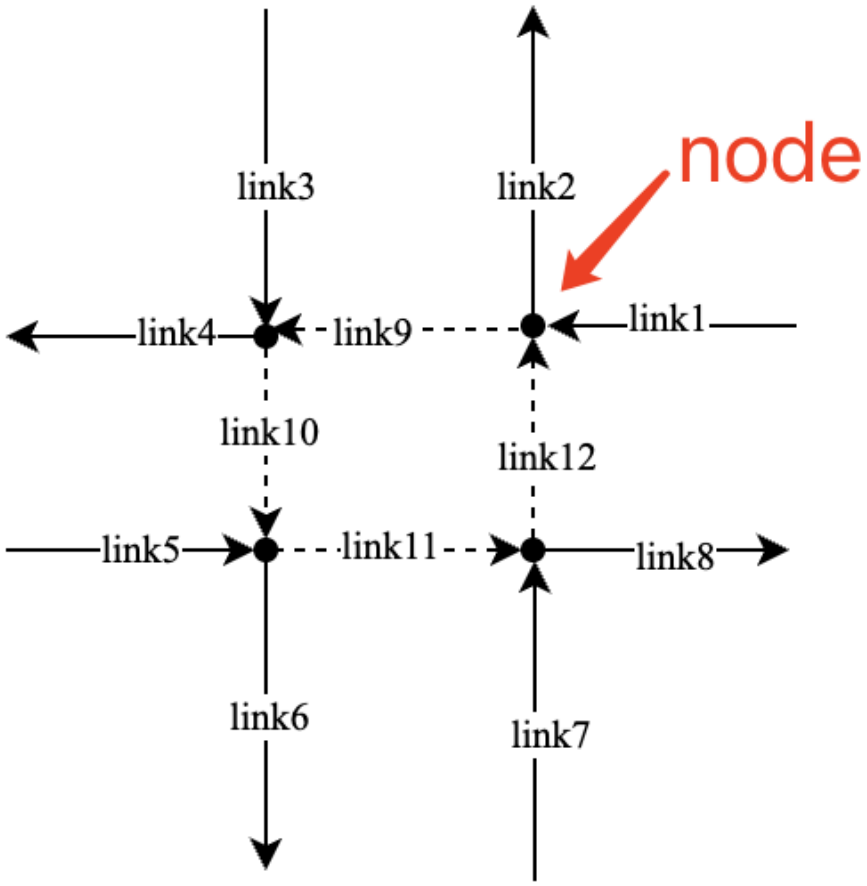
获取linkNode的拓扑关系

可以通过linkNode获取进入或者退出这个node的link对象或者linkId。如果node在tile的边界，有可能link对象没有补全，获取到的link对象不完整，因此推荐使用获取linkId。

```
// 获取进入的所有linkID集合
node->GetEnterLinkIDs();
// 获取出度的所有linkID集合
node->GetGetOutLinkIDs();
// 获取所在路口的所有进入linkID集合
node->GetEnterLinkIDsCross();
// 获取所在路口的所有出度linkID集合
node->GetOutLinkIDsCross();

// 获取进入的所有link集合
node->GetEnterElements();
// 获取出度的所有link集合
node->GetOutElements();
// 获取所在路口的所有进入link集合
node->GetEnterElementsCross();
// 获取所在路口的所有出度link集合
node->GetOutElementsCross();
```

非复杂路口，GetEnterLinkIDs和GetEnterLinkIDsCross、GetGetOutLinkIDs和GetOutLinkIDsCross、GetEnterElements和GetEnterElementsCross、GetOutElements和GetOutElementsCross的返回结果是一致的。

复杂路口是有区别，如下图所示红色箭头所指向的node：



node的GetEnterLinkIDs、GetEnterElements取的是link1、link12
node的GetGetOutLinkIDs、GetOutElements取的是link2、link9
node的GetEnterLinkIDsCross、GetEnterElementsCross取的是link1、link3、link5、link7
node的GetOutLinkIDsCross、GetOutElementsCross取的是link2、link4、link6、link8

Tile边界的LinkNode会重复存储

link会被tile打断，在tile边界会生成LinkNode，边界上的LinkNode会分别存储在两个tile中，属性相同但有不同的LinkNodeId，在判断LinkNode是否相等时可以根据LinkNodeId是否相等或者LinkNode坐标是否相等判断

2.3.1 车信LaneInfo

车信信息记录进入道路的车道信息，实地以及空中印刷了指示行车方向告知车辆及时切换车道的箭头信息，和指示车道具有特殊功能的信息。如带时间段的公交车道、转向可变的车道。

1. 车信获取

```
auto linkLayer = l_data_cube->GetSDLinkLayer();
// 获取所有link
auto links = linkLayer->GetLinkArr();
for (auto link : links) {
  // 获取link的车信信息
  auto laneInfos = link->GetGuidance(nerd::api::GuidanceType::kLaneMetaInfo);
  for (auto laneInfoPtr : laneInfos) {
    auto laneInfo = std::static_pointer_cast<const nerd::api::LaneInfoMeta>(laneInfoPtr);
    if (laneInfo == nullptr) {
        continue;
    }
  }
}
```

Note:
1）车信诱导信息与link相关连，因此必须通过link的GetGuidance接口获取才能获取到车信信息。
2）车信信息LaneInfoMeta继承与诱导基类IGuidanceInfo，基类中提供获取诱导信息的接口如下

- GuidanceType GetType()； // 返回诱导信息类型
- LinkIDType GetInLinkID()； // 返回进入linkId
- NodeIDType GetNodeID()； // 返回进入节点id
- std::vector<LinkIDType> GetPassLinkIDS(); // 返回退出linkId

> 针对车信诱导信息类型。仅GetType()，GetNodeID()接口是有效的。
> 车信是通过link来获取的，这条link就是进入link。
> 车信进入节点透过GetNodeID()获取。
> 车信退出link可以通过车信转向拓扑信息中可以获取。

2. 车信使用

车信结构由3部分组成：LaneInfoMeta, LaneTopo, LaneCond

LaneInfoMeta(车信信息 -> 继承于IGuidanceInfo诱导基类):

字段	类型	说明
lane_num	int	车道数
lane_infos	std::vector<LaneTrafficInfo>	车道通行信息(车道从左到右): 包含车道类型，是否公交车道，箭头信息
lane_num_l	uint8_t	左附加车道数
lane_num_r	uint8_t	右附加车道数
lane_topos	std::vector<LaneTopo>	车道转向拓扑

- **lane_info**: 车道通行信息，LaneTrafficInfo结构中包含了3个字段：
  - is_bus：是否是公交车道
  - lane_type：当前车道的车道类型，由多种车道类型枚举值通过与运算组成的一个uint32_t类型的数值(车道类型枚举参考 LaneExtensionType结构)
  - arrow：当前车道的车道箭头信息，由多个箭头类型枚举值通过与运算组成的组成的一个uint32_t类型的数值(箭头类型枚举参考 LaneArrowType 结构)
- **lane_topos**: 车道转向拓扑信息，详细结构参考LaneTopo类

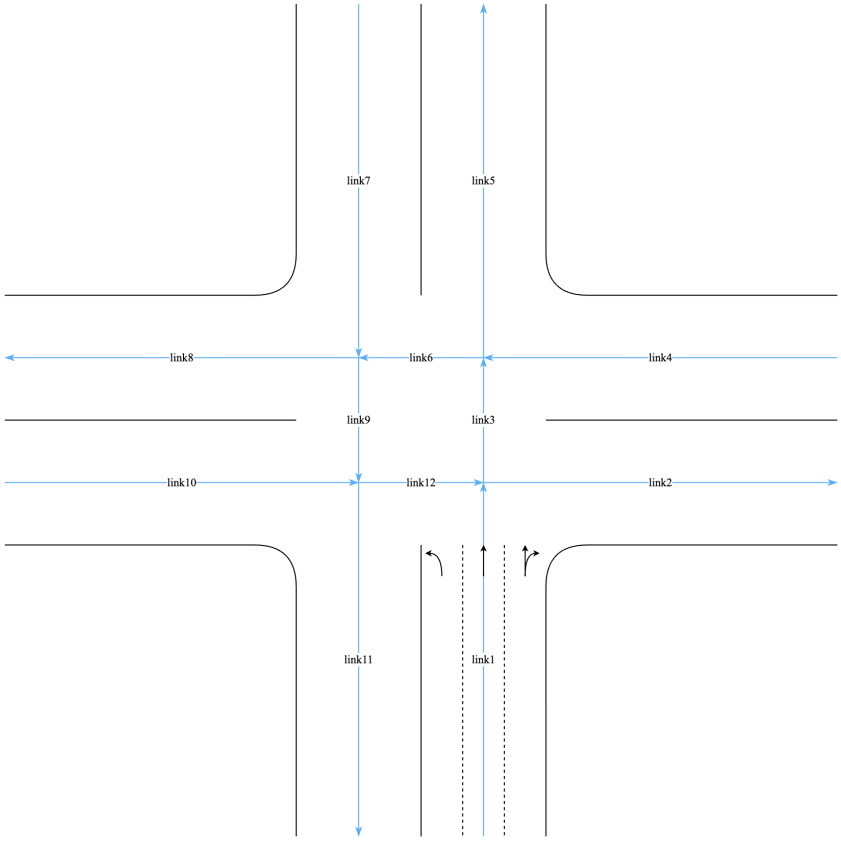
LaneTopo(车信转向拓扑):
车信转向拓扑可以理解成从一条link到另一条link的一个路径

字段	类型	说明
lane_arrow	LaneArrowType	转向箭头
lane_flag	uint16_t	生效车道，16bit从左到右表示车道是否生效
bus_flag	uint16_t	是否仅公交车生效，16bit从左到右表示是否生效
out_link_ids	std::vector<LinkIDType>	退出线号
lane_conds	std::vector<LaneCond>	车信条件

- **lane_arrow**： 车道箭头，车信转向拓扑信息中的车道箭头与LaneTrafficInfo不同，此处箭头指可能是直行，左转，左掉头，右转，右掉头，空白这5中类型中的一种。

LaneCond(车信条件):

字段	类型	说明
laneinfo_cond	uint16_t	生效车道，16bit从左到右表示每个车道是否生效
direction	LinkDirection	车道方向
date	std::string	时间段
vehicles	std::vector<VehicleType>	限制车辆类型
special_times	std::vector<SpecialTimeType>	特殊时间类型



假如存在左图的场景, 对照车信LaneInfoMeta, LaneTopo结构，分析link1处的车信:
LaneInfoMeta:
    lane_num = 3
    lane_num_l = 0
    lane_num_r = 0
    lane_infos中的车道通行与车道数是一一对应
    lane_infos[0]: 从左到右第一个车道
        arrow: 左转
        is_bus: false
        lane_type: 常规车道
    lane_infos[1]: 从左到右第二个车道
        arrow: 直行
        is_bus: false
        lane_type: 常规车道
    lane_infos[2]: 从左到右第三个车道
        arrow: 直行 + 右转
        is_bus: false
        lane_type: 常规车道

根据左图所示，link1可能的路径总共有3条：
1. link1->link2
2. link1->link3->link5
3. link1->link3->link6->link8
lane_topo的size为3。
lane_topos[0]:
    // 此处的箭头表示的就是link在当前路径下的转向信息
    lane_arrow: 左转
    lane_flag: 1000000000000000
    bus_flag: 0000000000000000
    out_links:[link3, link6, link8]
    lane_conds: 车信限制条件与每一个topo相关联
lane_topos[1]:
    // 此处的箭头表示的就是link在当前路径下的转向信息
    // 注意此处，直行的车道有两条，第2，3条车道均可直行
    lane_arrow: 直行
    lane_flag: 0110000000000000
    bus_flag: 0000000000000000
    out_links:[link3, link5]
    lane_conds: 车信限制条件与每一个topo相关联
lane_topos[2]:
    // 此处的箭头表示的就是link在当前路径下的转向信息
    lane_arrow: 右转
    lane_flag: 0010000000000000
    bus_flag: 0000000000000000
    out_links:[link2]
    lane_conds: 车信限制条件与每一个topo相关联

2.3.2 电子眼Camera

记录电子眼的类型、坐标、限速值、作用方向、拍摄位置、作用时间、作用车辆类型等信息。

1. 电子眼信息

字段	类型	说明
type	CameraType	类型，固定测速摄像头、闯红灯摄像头等
group_id	uint64_t	分组编号，区间测速起点、终点有效，标识同一组区间测速
coordinate	Coordinate	坐标
speed_limit	uint8_t	限速值
direction	LinkDirection	作用方向
position	ShootPosition	拍摄位置，车头、车尾或者未调查
start_date	string	开始作用时间，定义参考附录6.2
end_date	string	结束作用时间，定义参考附录6.2
vehicles	VehicleType[]	作用车辆类型
high_risk	bool	是否高危，违章次数>50为高危
scenes	CameraScene[]	摄像头场景，如斑马线前、道路左侧等
date	string	时间段，定义参考附录6.2

2. 电子眼信息获取

电子眼信息绑定在作用link上，可以通过link对象的GetGuidance方法获取，示例代码如下：

```
// 获取link上的电子眼信息
auto guidances = link->GetGuidance(nerd::api::GuidanceType::kCamera);
for (const auto &guidance : guidances) {
  auto camera = std::static_pointer_cast<const nerd::api::Camera>(guidance);
  // 获取电子眼类型
  auto camera_type = camera->type;
}
```

2.3.3 收费站TollStation

记录收费站的名称、作用方向、通道类型、通道收费方式等信息。

1. 收费站信息

字段	类型	说明
name	string	名称
direction	LinkDirection	作用方向
toll_means	uint8_t[]	通道收费方式，每个收费通道用7个Bit位组合表达，Bit为1表示支持，为0表示不支持： Bit0：现金 Bit1：ETC Bit2：微信 Bit3：支付宝 Bit4：其他支付 Bit5：免费 Bit6：未调查 比如第1个通道支持ETC，第2个通道支持现金/支付宝，第3、4通道支持现金/支付宝/其他支付可以表示为[2，9,25,25]
passage_ways	uint8_t[]	通道类型，每个通道类型用6个Bit位组合表达，Bit为1表示支持，为0表示不支持： Bit0：称重车道 Bit1：绿色车道 Bit2：通用车道 Bit3：预留 Bit4：预留 Bit5：预留 比如第一个收费口通道类型是通用车道，第二个收费口通道类型是称重+绿色通道可以表示为[4，3]
gate_flag	TollGateType	收费站出入口标识，值为出口、入口、出入口
name_eng	string	英文名称，暂未使用
separate_flag	TollSeparateType	分岔路标识，只针对经过收费站分叉道路进行标识、其他收费站设置为空
lane_group	uint8_t[]	通道方向分组，按照退出方向给车道进行分组，比如一共7车道，左边三个车道通往北京市区，右边四个车道通往北京机场，可表达为[3，4]，separate_flag为kExitMultiply时有效
group_name	string[]	针对lane group中的分组车道，每组通往的不同方向名称，按照上面的示例表达为[北京市区，北京机场]，separate_flag为kExitMultiply时有效

2. 收费站信息获取

收费站信息绑定在作用link上，可以通过link对象的GetGuidance方法获取，示例代码如下：

```
// 获取link上的收费站信息
auto guidances = link->GetGuidance(nerd::api::GuidanceType::kTollStation);
for (const auto &guidance : guidances) {
  auto toll_station = std::static_pointer_cast<const nerd::api::TollStation>(guidance);
  // 获取收费站名称
  auto name = toll_station->name;
}
```

2.3.4 警告牌WarningSign

记录警告牌的类型、提前预告距离、有效距离、作用方向、坐标等信息。

1. 警告牌信息

字段	类型	说明
trc_kind	TrafficSignType	警告牌类型，上陡坡、注意行人等警告标志
foretold_dist	uint16_t	提前预告距离，单位：米
control_dist	uint16_t	有效距离，单位：米
linkDirection	LinkDirection	作用方向
descript	string	文字说明，一般是文字，如“雨雾天气”、“急弯减速”
coordinate	Coordinate	坐标

2. 警告牌获取

警告牌绑定在作用link上，可以通过link对象的GetGuidance方法获取，示例代码如下：

```
// 获取link上的警告牌信息
auto guidances = link->GetGuidance(nerd::api::GuidanceType::kWarningSign);
for (const auto &guidance : guidances) {
  auto warning_sign = std::static_pointer_cast<const nerd::api::LinkWarningSign>(guidance);
  // 获取警告牌类型
  auto warning_type = warning_sign->trc_kind;
}
```

2.3.5 点限速PointSpeed

记录限速牌类型、限速值、提前预告距离、作用Link、作用方向、坐标等信息。

1. 点限速信息

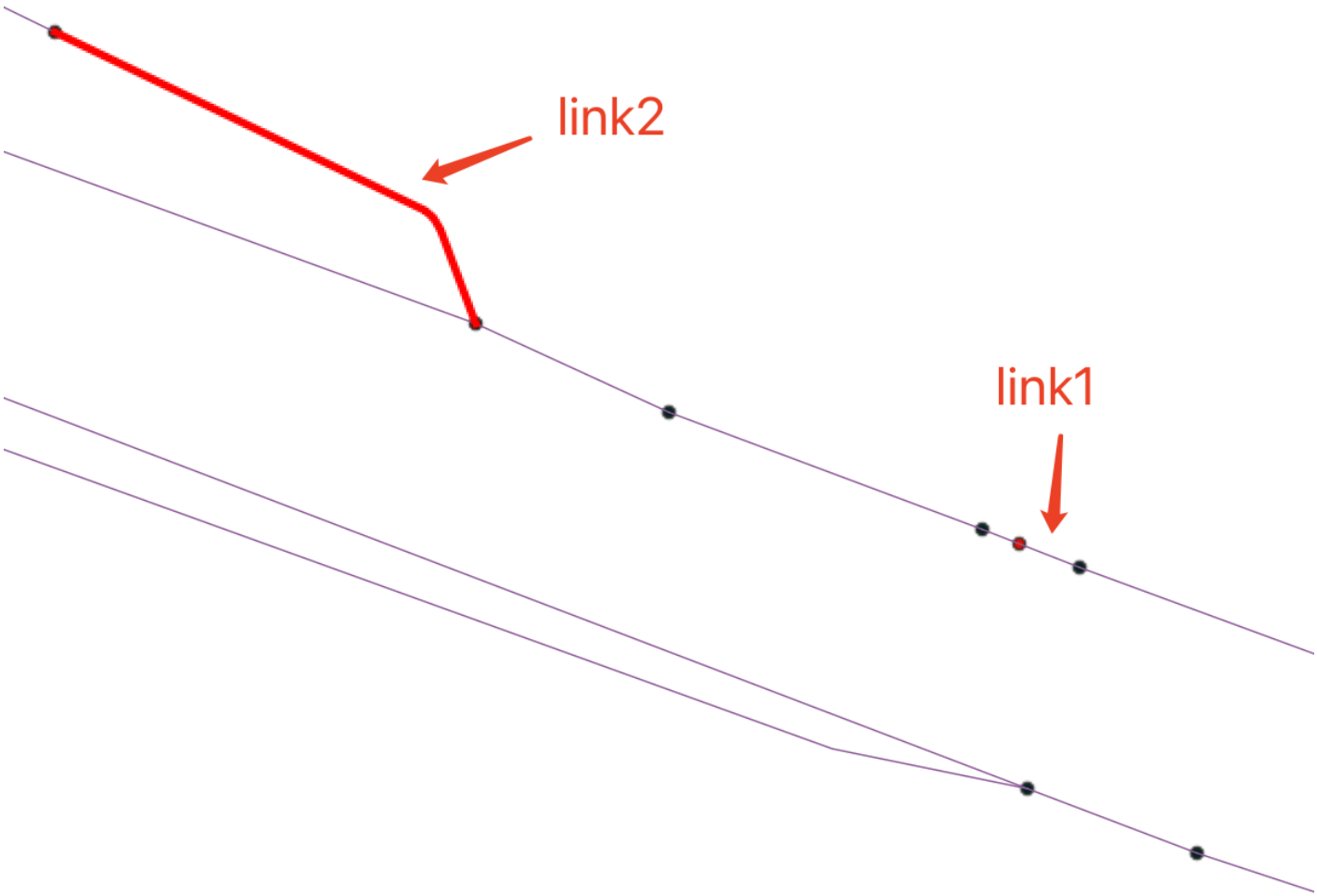
字段	类型	说明
type	PointSpeedType	限速牌类型，最高限速、最低限速、解除限速等类型
speed_limit	uint8_t	限速值，单位：km/h
pre_dist	uint16_t	提前预告距离，单位：米
coordinate	Coordinate	坐标
direction	LinkDirection	关联道路的方向
affect_link_id	LinkIDType	作用道路LinkId
affect_direction	LinkDirection	作用道路的方向
speed_scenes	PointSpeedScene[]	点限速场景，如隧道、匝道等
date	string	时间段限制，定义参考附录6.2
special_times	SpecialTimeType[]	特殊时间类型
nav_coordinate	Coordinate	引导坐标，当点限速距离作用道路过远，记录引导坐标便于使用
location	PointSpeedLocation	相对位置，如相对作用道路的左侧、右侧等
vehicles	VehicleType[]	作用车辆类型

2. 点限速获取

点限速绑定在关联link上，可以通过关联link对象的GetGuidance方法获取，示例代码如下：

```
// 获取link上的点限速信息
auto guidances = link->GetGuidance(nerd::api::GuidanceType::kPointSpeed);
for (const auto &guidance : guidances) {
  auto point_speed = std::static_pointer_cast<const nerd::api::PointSpeed>(guidance);
  // 获取点限速类型
  auto speed_type = point_speed->type;
}
```

3. 点限速关联link和作用link示意图



图片中黑色的点为LinkNode，红色的点为点限速，两个黑点之间的紫色线段为link。
图中的点限速绑定在关联link1上，但实际生效在作用link2上，常见于限速牌立在高速道路但实际控制的是后续匝道部分。

2.3.6 方向看板Board

记录方向看板的类型、看板名称、出口编号、退出link等信息。

1. 方向看板信息

字段	类型	说明
type	BoardType	看板类型，IC、高速方向、普通方向等类型
ic_exitnum	string	出口编号，可空
names	string[]	方向看板名称
name_directions	BoardNameDirection[]	名称属性，方向名称、出口名称、入口名称等，与names对应
names_eng	string[]	方向看板英文名称，已弃用
pass_link_ids	LinkIDType[]	退出link ID列表，通过函数GetPassLinkIDS()获取

2. 方向看板的获取

方向看板绑定在进入link上，可以通过link对象的GetGuidance方法获取，示例代码如下：

```
// 获取link上的方向看板信息
auto guidances = link->GetGuidance(nerd::api::GuidanceType::kBoard);
for (const auto &guidance : guidances) {
  auto board = std::static_pointer_cast<const nerd::api::Board>(guidance);
  // 获取看板类型
  auto board_type = board->type;
  // 获取看板的退出linkIds
  auto exit_links = board->GetPassLinkIDS();
}
```

2.3.7 长实线Lsl

记录长实线的类型、作用车道序号、退出link等信息。

1. 长实线信息

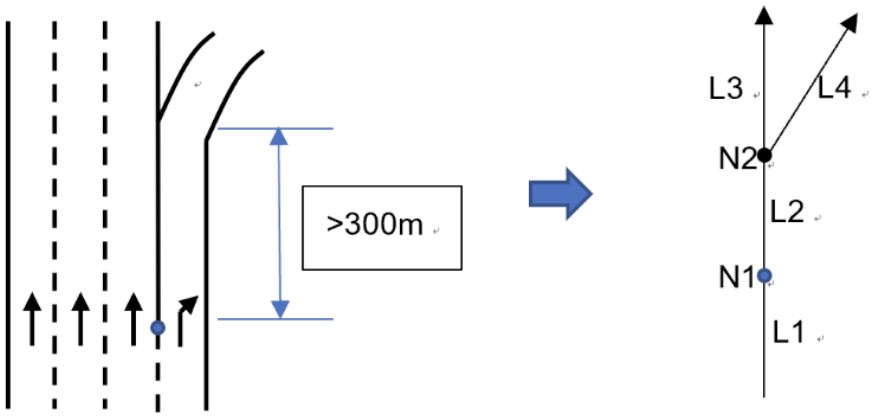
字段	类型	说明
seqnum	uint8_t	作用车道序号
lsl_type	LSLType	长实线类型，单实线、左虚右实、左实右虚等类型
pass_link_ids	LinkIDType[]	退出link ID列表，通过函数GetPassLinkIDS()获取

2. 长实线的获取

长实线绑定在node上，可以通过node对象的GetGuidance方法获取，示例代码如下：

```
// 获取node上的长实线信息
auto guidances = node->GetGuidance(nerd::api::GuidanceType::kLsl);
for (const auto &guidance : guidances) {
  auto lsl = std::static_pointer_cast<const nerd::api::LSL>(guidance);
  // 获取长实线类型
  auto board_type = lsl->lsl_type;
  // 获取长实线的退出linkIds
  auto exit_links = lsl->GetPassLinkIDS();
}
```

3. 长实线的数据示例



N1对象的GetGuidance方法获取可以取到两条长实线记录：

进入节点	作用车道序号	长实线类型	退出link IDs
N1	3	LSLType::kSingleSolid	L2L3
N1	3	LSLType::kSingleSolid	L2L4

2.3.8 坡度Slope